

Инструкция и образец

Производственная практика ПМ.08 — Разработка дизайна веб-приложений

7 семестр · Разработчик веб и мультимедийных приложений · Python / Flask / MySQL

09.02.07 — Информационные системы и программирование

ПМ.08 — Разработка дизайна веб-приложений

Учебные материалы СПО

Конференции.РФ

Создание учебного примера «Конференции.РФ» с пустой папки: модель MySQL, маршруты Python/Flask, шаблоны, дизайн и проверка пользовательского сценария. Весь разбор приведён в методичке; архив готового приложения не предоставляется.



Как превратить задачу в интерфейс

Учебный пример

Создадим «Конференции.РФ» самостоятельно: от пустой папки до страниц выбора площадки, регистрации, заявки, кабинета и администратора. Решение приведено прямо в методичке, отдельный готовый проект не требуется. Стек: Python, Flask, MySQL, HTML, CSS и JavaScript.

Пользователь и главное действие

Организатор хочет найти подходящий зал. На главной объясним назначение сервиса; в каталоге покажем вместимость и оснащение; в заявке дадим понятные поля; в кабинете обозначим состояние. Сначала набросайте каркасы страниц на бумаге: где заголовок, условия выбора, результат и главное действие. Цвет и декор выбираются после этой структуры.

Как работать с кодом

Создавайте указанные файлы самостоятельно. Фрагменты `app.py` и `design.css` добавляются сверху вниз в указанном порядке; шаблоны создаются отдельно. Подпись каждого фрагмента указывает место изменения. После законченного этапа проверяйте описанный результат. Полный серверный сценарий нужен для проверки дизайна, но основная цель ПМ.08 — понятный и доступный интерфейс.



Создать структуру своего примера

Пустая рабочая папка

Создайте папку `conference-learning`, внутри — `templates` и `static`. Пока в ней нет приложения. В этой папке появятся `app.py`, `schema.sql`, `rooms.json` и `requirements.txt`; HTML-шаблоны создадим в `templates`, дизайн — в `static/design.css` и `static/design.js`. Названия сохраняйте точно: Flask ищет `templates` и `static` автоматически.

Среда

Используйте Python 3.10 или новее и локальный MySQL. Проверка версии: Windows — `py -3 --version`, macOS — `python3 --version`. Для SQL можно использовать уже установленный MySQL Workbench. В терминале создаваемого проекта выполняются показанные далее команды.



Определить зависимости приложения

Создайте requirements.txt

```
Flask>=3.1,<4  
PyMySQL>=1.1,<2
```

Для чего нужны библиотеки

Flask связывает URL с функцией Python и выводит шаблоны. PyMySQL выполняет запросы к MySQL. werkzeug входит в зависимости Flask: его функции понадобятся для хеширования пароля. Установка этих зависимостей показана перед первым запуском.

Выделить отдельную базу MySQL

Создайте database.sql

```
CREATE DATABASE pm08_learning_2027 CHARACTER SET utf8mb4;  
CREATE USER 'pm08_learning'@'127.0.0.1' IDENTIFIED BY 'LocalPractice2027!';  
GRANT ALL PRIVILEGES ON pm08_learning_2027.* TO 'pm08_learning'@'127.0.0.1';
```

Выполнить SQL один раз

В локальном клиенте MySQL под администратором выполните SQL. Это отдельная учебная база и пользователь с правами только на неё. Если имя уже занято, выберите другое и используйте его далее; существующую базу не удаляйте.

Зачем отделять данные

Отдельная база исключает смешивание примера с вашей индивидуальной областью. Показанный пароль — вымышленное значение локального стенда. Здесь мы создаём только базу; таблицы опишем следующими фрагментами.

Спроектировать пользователей и площадки

Начало schema.sql: users и rooms

```
CREATE TABLE IF NOT EXISTS users (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  username VARCHAR(30) NOT NULL UNIQUE,  
  full_name VARCHAR(150) NOT NULL,  
  phone VARCHAR(18) NOT NULL,  
  email VARCHAR(254) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  is_admin BOOLEAN NOT NULL DEFAULT FALSE  
);  
CREATE TABLE IF NOT EXISTS rooms (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(150) NOT NULL,  
  category VARCHAR(20) NOT NULL,  
  city VARCHAR(100) NOT NULL,  
  address VARCHAR(200) NOT NULL,  
  capacity INT NOT NULL,  
  hourly_rate DECIMAL(10,2) NOT NULL,  
  image VARCHAR(200) NOT NULL DEFAULT '',  
  description TEXT NOT NULL,  
  equipment TEXT NOT NULL  
);
```

Связь данных с экраном

Пользователь вводит данные регистрации; площадка отображается карточкой каталога. UNIQUE запрещает повторные логины и почту. Пароль сохраняется только как хеш. В rooms храним характеристики, которые помогают выбрать помещение; category — техническое значение для фильтра.

Проверка структуры

Сопоставьте поля rooms с каркасом карточки: название, тип, город, вместимость, стоимость, описание, оснащение. image пока может быть пустой строкой: позже предусмотрим понятную заглушку.

Связать заявку, пользователя и отзыв

Продолжите schema.sql: applications и reviews

```
CREATE TABLE IF NOT EXISTS applications (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  user_id INT NOT NULL,  
  room_id INT NOT NULL,  
  event_name VARCHAR(120) NOT NULL,  
  event_date DATE NOT NULL,  
  event_time CHAR(5) NOT NULL,  
  participants INT NOT NULL,  
  payment VARCHAR(20) NOT NULL,  
  status VARCHAR(40) NOT NULL DEFAULT 'Новая',  
  FOREIGN KEY (user_id) REFERENCES users(id),  
  FOREIGN KEY (room_id) REFERENCES rooms(id)  
);  
  
CREATE TABLE IF NOT EXISTS reviews (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  application_id INT NOT NULL UNIQUE,  
  rating INT NOT NULL,  
  comment VARCHAR(500) NOT NULL,  
  FOREIGN KEY (application_id) REFERENCES applications(id)  
);
```

Почему именно эти связи

user_id и room_id связывают заявку с человеком и залом. FOREIGN KEY не допускает ссылки на отсутствующую запись. status содержит текст состояния. Один отзыв относится к одной заявке: UNIQUE на application_id запрещает повторный отзыв. DATE хранит дату как дату, а не текст.

Границы учебного примера

Заявка не является реальным платежом. Проверка пересекающихся бронирований в этом примере не реализована. Интерфейс должен описывать только фактически работающие действия.



Начать приложение Flask

Начало app.py: импорты и настройки

```
import json
import os
import re
import secrets
from datetime import date
from pathlib import Path

import click
import pymysql
from flask import Flask, abort, g, redirect, render_template, request, session, url_for
from werkzeug.security import check_password_hash, generate_password_hash

app = Flask(__name__)
app.secret_key = os.environ['FLASK_SECRET']
ROOT = Path(__file__).parent
TYPES = {'auditorium': 'Аудитория', 'coworking': 'Коворкинг', 'cinema': 'Кинозал'}
STATUSES = ('Новая', 'Мероприятие назначено', 'Завершено')
```

Разбор

Flask(__name__) создаёт приложение. FLASK_SECRET задаётся при запуске и используется для подписи сессии. Path указывает папку текущего файла. TYPES связывает технические категории с русскими подписями, STATUSES задаёт порядок состояний заявки. Пока не запускайте сервер: сначала дополним соединение и главную страницу.

Подключить Python к MySQL

Добавьте в app.py: db

```
def db():
    if 'db' not in g:
        g.db = pymysql.connect(
            host=os.environ['MYSQL_HOST'], port=int(os.environ['MYSQL_PORT']),
            user=os.environ['MYSQL_USER'], password=os.environ['MYSQL_PASSWORD'],
            database=os.environ['MYSQL_DATABASE'], charset='utf8mb4',
            cursorclass=pymysql.cursors.DictCursor, autocommit=True)
    return g.db
```

Добавьте в app.py: query

```
def query(sql, values=(), *, one=False, write=False):
    with db().cursor() as cursor:
        cursor.execute(sql, values)
        if write:
            return cursor.lastrowid
        return cursor.fetchone() if one else cursor.fetchall()
```

Добавьте в app.py: close_db

```
@app.teardown_appcontext
def close_db(error):
    connection = g.pop('db', None)
    if connection:
        connection.close()
```

Почему так

g хранит соединение только в рамках текущего запроса. DictCursor возвращает записи по именам полей. Все значения передаются отдельно от SQL через %s — пользовательский ввод не склеивается с запросом. write=True обозначает изменение данных; one=True — одну запись. teardown закрывает соединение после запроса.

Подготовить сессию и защиту формы

Добавьте в app.py: prepare_request

```
@app.before_request
def prepare_request():
    session.setdefault('csrf', secrets.token_urlsafe(24))
    g.user = query('SELECT * FROM users WHERE id=%s', (session['user_id'],), one=True) if 'user_id' in session else None
    if request.method == 'POST':
        token = request.form.get('csrf', '')
        if not secrets.compare_digest(token, session['csrf']):
            abort(400, 'Обновите форму: проверка сессии не пройдена.')
```

Добавьте в app.py: template_context

```
@app.context_processor
def template_context():
    return {'types': TYPES, 'statuses': STATUSES}
```

Добавьте в app.py: require_user

```
def require_user(admin=False):
    if not g.user:
        abort(401)
    if admin and not g.user['is_admin']:
        abort(403)
```

Связь сервера с интерфейсом

g.user определяет, какие действия доступны в навигации. Случайный csrf-токен передаётся скрытым полем и сравнивается при POST. context_processor передаёт подписи категорий и статусов во все шаблоны. require_user проверяет вход и роль на сервере: скрытая кнопка сама по себе не ограничивает доступ.

Создать команды для учебных данных

Добавьте в app.py: init_db

```
@app.cli.command('init-db')
def init_db():
    for statement in (ROOT / 'schema.sql').read_text().split(';'):
        if statement.strip():
            query(statement, write=True)
    query('INSERT IGNORE INTO users (username, full_name, phone, email, password_hash, is_admin) VALUES (%s,%s,%s,%s,%s,1)',
          ('Conf2027', 'Учебный Администратор', '8(900)000-00-00', 'admin@example.test', generate_password_hash('Demo77')), write=True)
    click.echo('Таблицы созданы. Учебный администратор: Conf2027 / Demo77.')
```

Добавьте в app.py: import_rooms

```
@app.cli.command('import-rooms')
@click.argument('filename', type=click.Path(exists=True, path_type=Path))
def import_rooms(filename):
    rows = json.loads(filename.read_text())
    for room in rows:
        query('INSERT INTO rooms (name,category,city,address,capacity,hourly_rate,image,description,equipment) VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s)',
              tuple(room[k] for k in ('name','category','city','address','capacity','hourly_rate','image','description','equipment'))), write=True)
    click.echo(f'Добавлено помещений: {len(rows)}. Повторный импорт добавит ещё записи.')
```

Что делают команды

init-db применяет созданный вами schema.sql и добавляет вымышленного администратора. import-rooms читает JSON и добавляет записи в rooms. Команды запускаются явно, поэтому понятно, когда создаются данные. Повторный импорт добавит дубликаты площадок: выполните его один раз в пустой базе. Импорт допускается только для вашего доверенного учебного JSON.

Записать данные демонстрационных площадок

Создайте rooms.json

```
[
  {"id": 1, "name": "Лофт «Балтика»", "category": "auditorium", "city": "Санкт-Петербург", "address": "Кожевенная линия, 30", "capacity": 120, "hourly_rate": 9500, "image": "", "description": "Исторический лофт с двумя экранами и профессиональным звуком.", "equipment": "2 экрана, проектор, звук, Wi-Fi"},
  {"id": 3, "name": "Коворкинг «Спектр»", "category": "coworking", "city": "Казань", "address": "ул. Петербургская, 52", "capacity": 70, "hourly_rate": 6100, "image": "", "description": "Гибкое пространство с рабочими зонами и местом для нетворкинга.", "equipment": "Wi-Fi, кухня, проектор, мобильная мебель"},
  {"id": 5, "name": "Кинозал «Вектор»", "category": "cinema", "city": "Нижний Новгород", "address": "ул. Рождественская, 18", "capacity": 42, "hourly_rate": 5700, "image": "", "description": "Тихий зал с амфитеатром для видеопозаов и презентаций.", "equipment": "Экран, Full HD проектор, 5.1 звук, Wi-Fi"}
]
```

Как читать JSON

Массив содержит три площадки. category принимает auditorium, coworking или cinema. image пока пуст: изображения мы не скачиваем вместе с приложением. Числа capacity и hourly_rate влияют на сравнение площадок. Измените одну вместимость в своём файле и объясните, как это должно отразиться на карточке.

Сделать общую оболочку страницы

Создайте templates/base.html

```
<!doctype html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{% block title %}Конференции.РФ{% endblock %}</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='design.css') }}">
  <script src="{{ url_for('static', filename='design.js') }}" defer></script>
</head>
<body>
<a class="skip-link" href="#main">Перейти к содержанию</a>
<header class="site-header">
  <a class="brand" href="{{ url_for('home') }}">Конференции.РФ</a>
  <nav aria-label="Главная навигация">
    <a href="/rooms">Помещения</a>
    {% if g.user %}
      <a href="{{ '/' if g.user.is_admin else '/dashboard' }}">Мои действия</a>
      <form action="/logout" method="post">
        <input type="hidden" name="csrf" value="{{ session.csrf }}">
        <button class="link-button">Выйти</button>
      </form>
    {% else %}
      <a href="/login">Войти</a>
      <a class="nav-primary" href="/register">Регистрация</a>
    {% endif %}
  </nav>
</header>
<main id="main">{% block content %}{% endblock %}</main>
<footer>Конференции.РФ · Учебный пример ПМ.08</footer>
</body>
</html>
```

Разбор структуры

lang и viewport задают язык и мобильный масштаб. block title и block content — места для содержимого дочернего шаблона. url_for строит адрес статических файлов. Навигация содержит пути будущих страниц; до реализации маршрутов они дадут 404 — это ожидаемый промежуточный результат. Кнопка выхода отправляет POST с csrf. Skip-link помогает перейти к содержанию клавиатурой.

Создать главную страницу

Добавьте в app.py: home

```
@app.route('/')
def home():
    return render_template('home.html')
```

Создайте templates/home.html

```
{% extends 'base.html' %}
{% block content %}
<section class="hero">
  <p class="eyebrow">Пространство для вашей идеи</p>
  <h1>Найдите площадку для встречи</h1>
  <p>Сравните вместимость и оснащение. Выберите помещение и отправьте заявку.</p>
  <a class="button button-primary" href="/rooms">Выбрать площадку</a>
</section>
<section class="explanation">
  <h2>От идеи до события</h2>
  <p>Выбор площадки → заявка → назначенное мероприятие → отзыв.</p>
  <p>Способ оплаты фиксируется в заявке. Реального платежа в учебном примере нет.</p>
</section>
{% endblock %}
```

Почему это решение помогает пользователю

extends использует общую оболочку. Главный заголовок объясняет задачу, текст уточняет действие, одна основная кнопка ведёт к выбору площадки. Пока оформление не добавлено, страница выглядит просто; это позволяет сначала проверить смысловую структуру.

Запустить созданную главную страницу в Windows

Выполните команды PowerShell в папке с app.py

```
py -3 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
$env:FLASK_SECRET="local-learning-secret-change-before-real-use"
$env:MYSQL_HOST="127.0.0.1"
$env:MYSQL_PORT="3306"
$env:MYSQL_DATABASE="pm08_learning_2027"
$env:MYSQL_USER="pm08_learning"
$env:MYSQL_PASSWORD="LocalPractice2027!"
.\.venv\Scripts\python.exe -m flask --app app init-db
.\.venv\Scripts\python.exe -m flask --app app import-rooms rooms.json
.\.venv\Scripts\python.exe -m flask --app app run --port 5019
```

Что проверить

Команды выполняются в созданной папке с app.py. Порт MySQL замените фактическим, если он отличается от 3306. Виртуальное окружение отделяет зависимости; прямой путь к Python не требует изменения ExecutionPolicy. Перейдите на <http://127.0.0.1:5019/> : виден текст главной. CSS и JS пока дадут 404 — создадим их далее. После остановки Ctrl+C повторно нужны только переменные окружения и последняя команда; импорт не повторяйте.

Запустить тот же пример в macOS

Выполните команды терминала в папке с app.py

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
export FLASK_SECRET=local-learning-secret-change-before-real-use
export MYSQL_HOST=127.0.0.1
export MYSQL_PORT=3306
export MYSQL_DATABASE=pm08_learning_2027
export MYSQL_USER=pm08_learning
export MYSQL_PASSWORD='LocalPractice2027!'
.venv/bin/python -m flask --app app init-db
.venv/bin/python -m flask --app app import-rooms rooms.json
.venv/bin/python -m flask --app app run --port 5019
```

Проверка первого результата

Это те же Python, Flask и MySQL; отличается синтаксис терминала. На <http://127.0.0.1:5019/> виден каркас главной страницы. Если нет соединения с MySQL, проверьте host, port, user и имя БД. FLASK_SECRET здесь демонстрационный для локальной работы; в реальной среде используйте отдельный случайный секрет.

Выбирать площадки по условиям

Добавьте в app.py: rooms

```
@app.route('/rooms')
def rooms():
    filters = {key: request.args.get(key, '').strip() for key in ('category', 'city', 'capacity')}
    sql, values = 'SELECT * FROM rooms WHERE 1=1', []
    if filters['category'] in TYPES:
        sql += ' AND category=%s'
        values.append(filters['category'])
    if filters['city']:
        sql += ' AND city=%s'
        values.append(filters['city'])
    if filters['capacity'].isdigit():
        sql += ' AND capacity>=%s'
        values.append(int(filters['capacity']))
    rows = query(sql + ' ORDER BY hourly_rate, id', tuple(values))
    return render_template('rooms.html', rooms=rows, filters=filters)
```

Разбор запроса

Фильтр использует GET: выбранные условия видны в URL. Python добавляет только известные условия, значения передаются параметрами. Сортировка по цене и id даёт предсказуемый порядок. `render_template` передаёт список `rooms` и введённые `filters` в HTML.

Промежуточная проверка

Сохраните `app.py`, остановите и снова запустите сервер последней командой. Не запускайте `init-db` и импорт повторно. Страница `/rooms` заработает после создания двух следующих шаблонов.

Показать фильтр, список и пустой результат

Создайте templates/rooms.html

```
{% extends 'base.html' %}
{% block title %}Каталог — Конференции.РФ{% endblock %}
{% block content %}
<h1>Помещения для вашей конференции</h1>
<form class="filter-panel" method="get">
  <label>Тип <select name="category"><option value="">Все типы</option>
    {% for key,label in types.items() %}<option value="{{ key }}" {% if filters.category==key %>selected{% endif %}>{{ label }}</option>{% endfor %}
  </select></label>
  <label>Город <input name="city" value="{{ filters.city }}"></label>
  <label>Не менее мест <input type="number" min="1" name="capacity" value="{{ filters.capacity
y }}"></label>
  <button class="button-primary">Показать</button>
  <a href="{{ url_for('rooms') }}">Сбросить</a>
</form>
<p role="status">Найдено: {{ rooms|length }}</p>
<div class="room-grid">
  {% for room in rooms %}{% include '_room_card.html' %}{% else %}
    <section class="empty-state"><h2>Подходящих помещений не нашлось</h2>
    <p>Измените условия или <a href="{{ url_for('rooms') }}">сбросьте фильтры</a>.</p>
  </section>
  {% endfor %}
</div>
{% endblock %}
```

Связь кода и состояния

Имена category, city, capacity совпадают с request.args. Атрибут selected сохраняет тип, value — город и вместимость. for выводит карточки; else показывает понятный пустой результат. Это не ошибка базы: запрос успешно выполнен, но записи не подошли.

Создать карточку площадки

Создайте templates/_room_card.html

```
<article class="room-card">
  {% if room.image %}
    
  {% else %}
    <div class="room-placeholder" aria-hidden="true">{{ types[room.category] }}</div>
  {% endif %}
  <div class="room-card-body">
    <p class="eyebrow">{{ room.city }} · {{ types[room.category] }}</p>
    <h2>{{ room.name }}</h2><p>{{ room.description }}</p>
    {% if room.equipment %}<p>Оснащение: {{ room.equipment }}</p>{% endif %}
    <p>До {{ room.capacity }} человек · {{ room.hourly_rate }} руб./ч</p>
    <a class="button button-primary" href="{{ '/apply/' ~ room.id if g.user else '/login' }}"
  >Выбрать</a>
  </div>
</article>
```

Почему карточка устроена так

Сначала пользователь считывает тип и город, затем название, описание, оснащение и стоимость. Главное действие — «Выбрать». Пустой image заменяем типографической заглушкой; фотографию можно добавить самостоятельно в static/img и указать относительный путь. Jinja экранирует текст, поэтому не применяйте |safe к данным.

Проверка

Перезапустите сервер. /rooms показывает три записи. Отфильтруйте тип, затем вместимость 1000: появляется сообщение и сброс. Вводите реальный город из JSON; в этом учебном решении он сопоставляется точно.

Создать единый компонент поля формы

Создайте templates/_field.html

```
{% macro field(name,label,type='text',hint='',required=true) %}
<div class="field">
  <label for="{{ name }}">{{ label }}{% if required %} *{% endif %}</label>
  <input id="{{ name }}" name="{{ name }}" type="{{ type }}"
    value="{{ ' ' if type=='password' else values.get(name,'') }}"
    {% if required %}required{% endif %}
    aria-describedby="{{ name }}-hint{% if errors.get(name) %} {{ name }}-error{% endif
%}">
  <small id="{{ name }}-hint">{{ hint }}</small>
  {% if errors.get(name) %}<span class="field-error" id="{{ name }}-error">{{ errors[name] }}
</span>{% endif %}
</div>
{% endmacro %}
```

Для чего нужен macro

field повторно использует одну структуру: подпись, поле, подсказка, ошибка. for совпадает с id; name передаёт значение Flask. values сохраняет введённый текст после ошибки. Пароль специально не возвращается в HTML. aria-describedby связывает подсказку и сообщение с полем. Это компонент интерфейса, который предотвращает различия между одинаковыми полями.

Проверить регистрацию на сервере

Добавьте в app.py: register

```
@app.route('/register', methods=['GET', 'POST'])
def register():
    values = request.form if request.method == 'POST' else {}
    errors = {}
    if request.method == 'POST':
        rules = {'username': (r'[A-Za-z0-9]{6,30}', 'Латиница и цифры: 6-30 символов.'),
                  'full_name': (r'[A-Яа-яЁё]+(?: [A-Яа-яЁё]+)?', 'Укажите ФИО кириллицей.'),
                  'phone': (r'8\(\d{3}\)\d{3}-\d{2}-\d{2}', 'Формат: 8(XXX)XXX-XX-XX.'),
                  'email': (r'^[^\s@]+@[^\s@]+\.[^\s@]+', 'Укажите адрес почты.')}
        for key, (pattern, message) in rules.items():
            if not re.fullmatch(pattern, values.get(key, '')):
                errors[key] = message
        password = values.get('password', '')
        if not 8 <= len(password) <= 128:
            errors['password'] = 'Пароль: от 8 до 128 символов.'
        if values.get('confirm') != password:
            errors['confirm'] = 'Пароли не совпадают.'
        if not errors:
            try:
                query('INSERT INTO users (username,full_name,phone,email,password_hash) VALUE
S (%s,%s,%s,%s,%s)',
                    tuple(values[k] for k in rules) + (generate_password_hash(password),),
                write=True)
                return redirect(url_for('login'))
            except pymysql.err.IntegrityError:
                errors['username'] = 'Такой логин или адрес почты уже используется.'
        return render_template('register.html', values=values, errors=errors)
```

Разбор

Правила проверяют каждое поле и заполняют словарь errors. Пароль должен иметь 8–128 символов, подтверждение совпасть. INSERT выполняется только при отсутствии ошибок; пароль хешируется. Уникальность окончательно проверяет MySQL. При ошибке снова выводим форму с values и errors, при успехе перенаправляем на вход.

Создать страницу регистрации

Создайте templates/register.html

```
{% extends 'base.html' %}
{% from '_field.html' import field with context %}
{% block title %}Регистрация — Конференции.РФ{% endblock %}
{% block content %}
<section class="form-card"><h1>Создайте учётную запись</h1><p>Все поля обязательны.</p>
<form method="post" class="stack-form" novalidate>
  <input type="hidden" name="csrf" value="{{ session.csrf }}">
  {{ field('username', 'Логин', hint='Латиница и цифры: 6-30 символов') }}
  {{ field('full_name', 'ФИО', hint='Кириллица и пробелы') }}
  {{ field('phone', 'Телефон', hint='8 (XXX) XXX-XX-XX') }}
  {{ field('email', 'Электронная почта', 'email') }}
  {{ field('password', 'Пароль', 'password', 'От 8 до 128 символов') }}
  {{ field('confirm', 'Повторите пароль', 'password') }}
  <button class="button-primary">Создать пользователя</button>
</form></section>
{% endblock %}
```

Связь формы и Python

with context передаёт компоненту values и errors. Шесть вызовов field создают шесть обязательных полей. novalidate позволяет увидеть единые серверные сообщения: атрибут required остаётся описанием обязательности. csrf не входит в пользовательский счётчик.

Проверка

Перезапустите сервер, отправьте пустую регистрацию: под неверными полями появляется текст. После реализации входа и кабинета зарегистрируйте pm08student / Practice2027, ФИО «Практика Пользователь», телефон 8(900)123-45-67, почта student@example.test. Эти данные вымышленные.

Реализовать вход и выход

Добавьте в app.py: login

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    errors = {}
    if request.method == 'POST':
        user = query('SELECT * FROM users WHERE username=%s', (request.form.get('username', ''),), one=True)
        if user and check_password_hash(user['password_hash'], request.form.get('password', '')):
            session.clear()
            session['user_id'] = user['id']
            return redirect(url_for('admin' if user['is_admin'] else 'dashboard'))
        errors['password'] = 'Логин или пароль неверны.'
    return render_template('login.html', errors=errors, values={})
```

Добавьте в app.py: logout

```
@app.post('/logout')
def logout():
    session.clear()
    return redirect(url_for('home'))
```

Создайте templates/login.html

```
{% extends 'base.html' %}
{% from '_field.html' import field with context %}
{% block title %}Вход — Конференции.РФ{% endblock %}
{% block content %}
<section class="form-card"><h1>Войти в сервис</h1>
<form method="post" class="stack-form" novalidate>
    <input type="hidden" name="csrf" value="{{ session.csrf }}">
    {{ field('username', 'Логин') }}{{ field('password', 'Пароль', 'password') }}
    <button class="button-primary">Войти</button>
</form></section>
{% endblock %}
```



Почему это решение корректнее сравнения паролей

`check_password_hash` сравнивает введенный пароль с хешем. При входе старая сессия очищается; в новой хранится только `id`. Администратор направляется в панель, пользователь — в кабинет. Logout выполняется через POST; следующий запрос создаст новый csrf. Кабинет реализуем ниже.

Проверить заявку и сохранить её

Добавьте в app.py: apply

```
@app.route('/apply/<int:room_id>', methods=['GET', 'POST'])
def apply(room_id):
    require_user()
    room = query('SELECT * FROM rooms WHERE id=%s', (room_id,), one=True)
    if not room:
        abort(404)
    values, errors = request.form if request.method == 'POST' else {}, {}
    if request.method == 'POST':
        if not 3 <= len(values.get('event_name', '').strip()) <= 120:
            errors['event_name'] = 'Название: 3-120 символов.'
        try:
            if date.fromisoformat(values.get('event_date', '')) < date.today():
                raise ValueError
        except ValueError:
            errors['event_date'] = 'Укажите сегодняшнюю или будущую дату.'
        if not re.fullmatch(r'(?:[01]\d|2[0-3]):[0-5]\d', values.get('event_time', '')):
            errors['event_time'] = 'Укажите время.'
        if not values.get('participants', '').isdigit() or not 1 <= int(values.get('participants', '0')) <= room['capacity']:
            errors['participants'] = 'Число участников должно соответствовать вместимости.'
        if values.get('payment') not in ('Карта', 'СБП', 'Наличные'):
            errors['payment'] = 'Выберите способ оплаты.'
        if not errors:
            query('INSERT INTO applications (user_id,room_id,event_name,event_date,event_time,participants,payment) VALUES (%s,%s,%s,%s,%s,%s,%s)',
                (g.user['id'], room_id, values['event_name'].strip(), values['event_date'],
                 values['event_time'], int(values['participants']), values['payment']), write=True)
            return redirect(url_for('dashboard'))
    return render_template('apply.html', room=room, values=values, errors=errors)
```

Что обеспечивает сервер

room_id определяет площадку и её вместимость. Проверяем название, дату, время, число участников и оплату; чужой user_id из формы не принимаем, используем g.user.id. При ошибке возвращаем форму с объяснением, при успехе показываем кабинет. Оплата — значение заявки, реального платежа нет.

Создать интерфейс заявки

Создайте templates/apply.html

```
{% extends 'base.html' %}
{% from '_field.html' import field with context %}
{% block title %}Заявка — Конференции.РФ{% endblock %}
{% block content %}
<section class="form-card"><h1>Заявка: {{ room.name }}</h1><p>До {{ room.capacity }} участник
ов.</p>
<form method="post" class="stack-form" novalidate>
  <input type="hidden" name="csrf" value="{{ session.csrf }}">
  {{ field('event_name', 'Название мероприятия', hint='3-120 символов') }}
  {{ field('event_date', 'Дата', 'date') }}{{ field('event_time', 'Время', 'time') }}
  {{ field('participants', 'Участники', 'number') }}
  <div class="field"><label for="payment">Способ оплаты *</label>
    <select id="payment" name="payment" required>
      <option value="">Выберите</option>
      {% for item in ['Карта', 'СБП', 'Наличные'] %}<option {% if values.get('payment')==item %
}selected{% endif %}>{{ item }}</option>{% endfor %}
    </select>
    {% if errors.payment %}<span class="field-error">{{ errors.payment }}</span>{% endif %}
  </div>
  <button class="button-primary">Отправить заявку</button>
</form></section>
{% endblock %}
```

Почему такой порядок полей

Заголовок подтверждает выбранную площадку. Далее вводятся название, дата, время, число людей и способ оплаты. Компонент field делает оформление и ошибки такими же, как в регистрации. Select оплаты начинается с явного «Выберите». Здесь пять обязательных ответов.

Проверка

После создания кабинета войдите как пользователь, выберите площадку и отправьте заявку с числом участников больше вместимости. Исправьте ошибку и повторите. До реализации кабинета успешный POST не проверяйте.

Показать пользователю его заявки

Добавьте в app.py: dashboard

```
@app.route('/dashboard')
def dashboard():
    require_user()
    rows = query('SELECT a.*,r.name AS room_name,v.rating,v.comment FROM applications a JOIN
rooms r ON r.id=a.room_id LEFT JOIN reviews v ON v.application_id=a.id WHERE a.user_id=%s ORD
ER BY a.id DESC', (g.user['id'],))
    return render_template('dashboard.html', applications=rows)
```

Создайте templates/dashboard.html

```
{% extends 'base.html' %}
{% block content %}
<h1>Мои заявки</h1>
{% for item in applications %}
<article class="application-card">
  <h2>{{ item.event_name }}</h2><p>{{ item.room_name }} · {{ item.event_date }} {{ item.event
_time }}</p>
  <p class="status">{{ item.status }}</p>
  {% if item.rating %}<p>Ваш отзыв: {{ item.rating }}/5 — {{ item.comment }}</p>
  {% elif item.status=='Завершено' %}
  <form method="post" action="{{ url_for('review', application_id=item.id) }}">
    <input type="hidden" name="csrf" value="{{ session.csrf }}">
    <label>Оценка <select name="rating">{% for n in range(1,6) %}<option>{{ n }}</option>{% e
ndfor %}</select></label>
    <label>Отзыв <textarea name="comment" required maxlength="500"></textarea></label>
    <button class="button-primary">Сохранить отзыв</button>
  </form>
  {% endif %}
</article>
{% else %}<p>Заявок пока нет. <a href="{{ url_for('rooms') }}">Выберите площадку</a>.</p>{% e
ndfor %}
{% endblock %}
```

Приватность и понятный статус

WHERE a.user_id ограничивает список текущим пользователем. JOIN добавляет название площадки, LEFT JOIN — возможный отзыв. Статус показывается текстом. Отзыв появляется только после завершения; это условие интерфейса дополнительно проверяется сервером.

Проверка

Теперь можно пройти регистрацию, вход, выбор площадки и успешную заявку. В кабинете видна только своя запись со статусом «Новая». Пустое состояние даёт переход в каталог.

Отделить роль администратора

Добавьте в app.py: admin

```
@app.route('/admin')
def admin():
    require_user(admin=True)
    rows = query('SELECT a.*,r.name AS room_name,u.full_name FROM applications a JOIN rooms r
ON r.id=a.room_id JOIN users u ON u.id=a.user_id ORDER BY a.id DESC')
    return render_template('admin.html', applications=rows)
```

Добавьте в app.py: change_status

```
@app.post('/status/<int:application_id>')
def change_status(application_id):
    require_user(admin=True)
    row = query('SELECT status FROM applications WHERE id=%s', (application_id,), one=True)
    if not row:
        abort(404)
    next_status = request.form.get('status', '')
    current = STATUSES.index(row['status'])
    if current == 2 or next_status != STATUSES[current + 1]:
        abort(400, 'Допустим только следующий статус.')
    query('UPDATE applications SET status=%s WHERE id=%s', (next_status, application_id), write=True)
    return redirect(url_for('admin'))
```

Почему недостаточно скрыть кнопку

`require_user(admin=True)` проверяет роль на сервере при просмотре и изменении. Разрешён только следующий статус: Новая → Мероприятие назначено → Завершено. Чужой URL и неверное значение статуса не обходят проверку. Обычный пользователь получает 403 на `/admin`.

Создать панель изменения статуса

Создайте templates/admin.html

```
{% extends 'base.html' %}
{% block content %}
<h1>Панель администратора</h1>
{% for item in applications %}
<article class="application-card"><h2>{{ item.event_name }}</h2>
<p>{{ item.full_name }} · {{ item.room_name }} · {{ item.event_date }} {{ item.event_time }}<
/p>
<p class="status">{{ item.status }}</p>
{% if item.status != 'Завершено' %}
<form method="post" action="{{ url_for('change_status', application_id=item.id) }}">
  <input type="hidden" name="csrf" value="{{ session.csrf }}">
  <input type="hidden" name="status" value="{{ statuses[statuses.index(item.status)+1] }}">
  <button class="button-primary">{{ statuses[statuses.index(item.status)+1] }}</button>
</form>
{% endif %}</article>
{% else %}<p>Новых заявок пока нет.</p>{% endfor %}
{% endblock %}
```

Решение интерфейса

Показываем событие, организатора, площадку, дату и текущий статус. Кнопка прямо называет следующий результат; скрытое поле передаёт его серверу. У завершённой заявки кнопки перехода нет.

Проверка

Выйдите, войдите как Conf2027 / Demo77. У созданной заявки выполните оба перехода. При повторном открытии панели состояние сохранено в MySQL. Данные администратора вымышленные.

Сохранить отзыв после завершения

Добавьте в app.py: review

```
@app.post('/review/<int:application_id>')
def review(application_id):
    require_user()
    row = query('SELECT * FROM applications WHERE id=%s AND user_id=%s', (application_id, g.user['id']), one=True)
    if not row:
        abort(404)
    rating, comment = request.form.get('rating', ''), request.form.get('comment', '').strip()
    if row['status'] != 'Завершено' or rating not in ('1','2','3','4','5') or not 1 <= len(comment) <= 500:
        abort(400, 'Отзыв: завершённая своя заявка, оценка 1-5 и текст 1-500 символов.')
    try:
        query('INSERT INTO reviews (application_id,rating,comment) VALUES (%s,%s,%s)', (application_id, int(rating), comment), write=True)
    except pymysql.err.IntegrityError:
        abort(400, 'Отзыв уже оставлен.')
    return redirect(url_for('dashboard'))
```

Проверка условий

Запрос ищет одновременно id заявки и id пользователя. Чужая заявка не найдётся. Сервер проверяет завершённый статус, оценку 1–5 и длину текста. UNIQUE в reviews исключает повторный отзыв даже при повторной отправке.

Проверка полного сценария

Войдите как pm08student, оставьте оценку и отзыв после завершения. Затем войдите как другой учебный пользователь: чужая запись не отображается. На этом серверный каркас создан вами; дальше улучшаем его дизайн.

Задать палитру и читаемую типографику

Последовательный фрагмент static/design.css

```
:root {
  --accent: #0d47a1;
  --text: #162b48;
  --error: #a81727;
  --space: 8px;
}
* { box-sizing: border-box; }
body { margin: 0; color: var(--text); background: #f3f6fb; font: 16px/1.6 system-ui, sans-serif; }
a { color: var(--accent); }
h1 { font-size: clamp(28px, 4vw, 48px); line-height: 1.15; }
h2 { font-size: 24px; line-height: 1.3; }
main { max-width: 1160px; margin: 40px auto; padding: 0 24px; }
```

Место и смысл

Создайте static/design.css. Переменные задают общие цвета и шаг. box-sizing включает отступы в размер, clamp изменяет заголовок без резкого скачка. Синий #0d47a1 с белым имеет контраст около 8,6:1.

Проверка

Обновите соответствующую страницу. Проверьте Network: design.css имеет статус 200. Сравните результат с каркасом, пройдите клавиатурой и повторите действие. Для мобильного варианта используйте 390×844.

Оформить навигацию и главное объяснение

Последовательный фрагмент static/design.css

```
.site-header { display: flex; justify-content: space-between; align-items: center; gap: 24px;
padding: 24px max(24px, calc((100vw - 1112px)/2)); background: white; }
.brand { font-size: 22px; font-weight: 800; text-decoration: none; }
nav { display: flex; flex-wrap: wrap; align-items: center; gap: 20px; }
nav form { margin: 0; }
footer { margin-top: 40px; padding: 24px; text-align: center; background: #162b48; color: white; }
.hero { padding: 56px; background: #e2ebfa; border-radius: 20px; }
.hero h1 { max-width: 650px; }
.eyebrow { font-weight: 700; letter-spacing: .04em; }
.explanation { margin-top: 32px; padding: 24px; }
```

Место и смысл

Добавьте следующий блок в конец static/design.css. Оболочка и навигация используют flex. Каркас hero выделяет назначение сервиса; заголовок ограничен удобной длиной строки.

Проверка

Обновите соответствующую страницу. Проверьте Network: design.css имеет статус 200. Сравните результат с каркасом, пройдите клавиатурой и повторите действие. Для мобильного варианта используйте 390×844.

Различить главное действие и клавиатурный фокус

Последовательный фрагмент static/design.css

```
.button, button, .nav-primary { display: inline-block; min-height: 44px; border: 0; border-radius: 8px; padding: 12px 20px; font: inherit; text-decoration: none; cursor: pointer; }
.button-primary, .nav-primary { background: var(--accent); color: white; }
.link-button { padding: 0; background: transparent; color: var(--accent); }
:focus-visible { outline: 3px solid var(--accent); outline-offset: 4px; }
.skip-link { position: absolute; top: -100px; left: 16px; padding: 12px; background: white; }
.skip-link:focus { top: 8px; z-index: 1; }
```

Место и смысл

Добавьте следующий блок в конец static/design.css. Кнопка имеет область нажатия не меньше 44 px. Основное действие синее, выход остаётся текстовым. focus-visible показывает текущий элемент при Tab; skip-link появляется при фокусе.

Проверка

Обновите соответствующую страницу. Проверьте Network: design.css имеет статус 200. Сравните результат с каркасом, пройдите клавиатурой и повторите действие. Для мобильного варианта используйте 390×844.

Оформить поля, ошибку и обратную связь

Последовательный фрагмент static/design.css

```
.form-card { max-width: 640px; margin: auto; padding: 32px; border-radius: 16px; background: white; }
.form-card h1 { font-size: 32px; }
.stack-form { display: grid; gap: calc(2 * var(--space)); }
.field { display: grid; gap: 4px; }
label { font-weight: 600; }
input, select, textarea { display: block; width: 100%; min-height: 44px; padding: 10px 12px; border: 1px solid #72829b; border-radius: 6px; color: inherit; font: inherit; background: white; }
small { font-size: 13px; color: #465873; }
.field-error { font-size: 14px; color: var(--error); }
[aria-invalid="true"] { border: 2px solid var(--error); }
.design-summary { padding: 12px 16px; margin: 0; background: #e9f0fa; border-left: 4px solid var(--accent); }
.design-summary strong { display: block; }
```

Место и смысл

Добавьте следующий блок в конец static/design.css. Форма построена сеткой. Подпись, подсказка и ошибка рядом с вводом; aria-invalid усиливает рамку. design-summary появится после создания JavaScript.

Проверка

Обновите соответствующую страницу. Проверьте Network: design.css имеет статус 200. Сравните результат с каркасом, пройдите клавиатурой и повторите действие. Для мобильного варианта используйте 390×844.

Построить карточки и состояния кабинета

Последовательный фрагмент static/design.css

```
.filter-panel { display: grid; grid-template-columns: repeat(3,minmax(0,1fr)) auto auto; gap: 16px; align-items: end; padding: 24px; background: white; border-radius: 12px; }
.room-grid { display: grid; grid-template-columns: repeat(3,minmax(0,1fr)); gap: 24px; }
.room-card { display: flex; flex-direction: column; border-radius: 16px; background: white; overflow: hidden; }
.room-card img { width: 100%; aspect-ratio: 16/10; object-fit: cover; }
.room-placeholder { display: grid; place-items: center; aspect-ratio: 16/10; font-size: 28px; font-weight: 800; color: #0d47a1; background: #e2ebfa; }
.room-card-body { display: flex; flex: 1; flex-direction: column; padding: 24px; }
.room-card-body .button { margin-top: auto; align-self: start; }
.room-card h2, .application-card h2 { overflow-wrap: anywhere; }
.empty-state { grid-column: 1/-1; padding: 32px; background: white; border-radius: 12px; }
.application-card { margin-bottom: 24px; padding: 24px; background: white; border-radius: 12px; }
.application-card form { max-width: 500px; display: grid; gap: 16px; }
.status { display: inline-block; padding: 4px 12px; border-radius: 20px; background: #e2ebfa; font-weight: 700; }
```

Место и смысл

Добавьте следующий блок в конец static/design.css. Каталог строится grid, карточка — flex. Основное действие остаётся внизу, длинный заголовок переносится. Пустое состояние занимает всю сетку.

Проверка

Обновите соответствующую страницу. Проверьте Network: design.css имеет статус 200. Сравните результат с каркасом, пройдите клавиатурой и повторите действие. Для мобильного варианта используйте 390×844.

Перестроить страницы для узкого экрана

Последовательный фрагмент static/design.css

```
@media (max-width: 700px) {  
  .site-header { align-items: start; flex-direction: column; padding: 20px; }  
  nav { gap: 16px; }  
  main { margin: 24px auto; padding: 0 16px; }  
  .hero { padding: 24px; }  
  .form-card { padding: 20px; }  
  .filter-panel, .room-grid { grid-template-columns: minmax(0,1fr); }  
}
```

Место и смысл

Добавьте следующий блок в конец static/design.css. На ширине до 700 px навигация и фильтры перестраиваются, карточки занимают одну колонку. `minmax(0,1fr)` позволяет колонке сжиматься. Не скрывайте переполнение вместо исправления причины.

Проверка

Обновите соответствующую страницу. Проверьте Network: design.css имеет статус 200. Сравните результат с каркасом, пройдите клавиатурой и повторите действие. Для мобильного варианта используйте 390×844.

Уважать предпочтение уменьшенного движения

Последовательный фрагмент static/design.css

```
.design-summary { animation: appear .18s ease-out; }
@keyframes appear { from { opacity: 0; transform: translateY(4px); } to { opacity: 1; transform: none; } }
@media (prefers-reduced-motion: reduce) {
  *, ::before, ::after { animation: none !important; transition: none !important; }
}
```

Место и смысл

Добавьте следующий блок в конец static/design.css. Анимация короткая и не задерживает ввод. prefers-reduced-motion отключает её при пользовательском предпочтении.

Проверка

Обновите соответствующую страницу. Проверьте Network: design.css имеет статус 200. Сравните результат с каркасом, пройдите клавиатурой и повторите действие. Для мобильного варианта используйте 390×844.

Добавить подсказку заполнения формы

Создайте static/design.js: первая часть

```
// ПМ.08: подсказка сопровождает основную форму, сервер проверяет данные.
'use strict';
const designForm = document.querySelector('.stack-form');
if (designForm) {
  const allFields = [...designForm.querySelectorAll('input, select, textarea')];
  const fields = allFields.filter((field, index) => field.type !== 'hidden' && field.required
    && (field.type !== 'radio' || allFields.findIndex(other => other.name === field.name) ===
index));
  const summary = document.createElement('p');
  summary.className = 'design-summary';
  summary.setAttribute('role', 'status');
  const heading = document.createElement('strong');
  heading.textContent = 'Заполнение формы';
  const counter = document.createElement('span');
  summary.append(heading, counter);
  designForm.prepend(summary);
  function updateSummary() {
    const filled = fields.filter(field => field.type === 'radio'
      ? allFields.some(other => other.name === field.name && other.checked)
      : field.value.trim() !== '').length;
    counter.textContent = `Заполнено ${filled} из ${fields.length} обязательных полей. Провер
ка выполняется при отправке.`;
  }
  designForm.addEventListener('input', updateSummary);
  designForm.addEventListener('change', updateSummary);
  updateSummary();
}
```

Разбор

Выбираем `.stack-form`, отбираем обязательные поля, исключаем скрытый csrf. Создаём подсказку через `textContent` и обновляем на `input/change`. Счётчик сообщает факт заполнения, а не правильность данных; окончательно проверяет Python. Работа с группами radio оставлена как приём для самостоятельного варианта.

Проверка

В регистрации 0 из 6 → введите логин → 1 из 6 → очистите → 0 из 6. В заявке пять полей; выбор оплаты добавляет один ответ. На странице без формы if предотвращает ошибку.

Связать сообщение ошибки с полем

Продолжите static/design.js: вторая часть

```
// Связываем видимые серверные ошибки с соответствующими полями.
const errors = document.querySelectorAll('.field-error');
let firstInvalid = null;
errors.forEach((error, index) => {
  if (!error.id) error.id = `design-error-${index}`;
  const group = error.closest('.field');
  // Поле и его ошибка находятся внутри одной группы .field.
  const input = group ? group.querySelector('input, select, textarea') : null;
  if (!input || !input.matches('input, select, textarea')) return;
  input.setAttribute('aria-invalid', 'true');
  const ids = new Set((input.getAttribute('aria-describedby') || '').split(/\s+/).filter(Boolean));
  ids.add(error.id);
  input.setAttribute('aria-describedby', [...ids].join(' '));
  firstInvalid ||= input;
});
if (firstInvalid) firstInvalid.focus();
```

Разбор

Находим сообщения `.field-error`, выбираем поле внутри `.field`. `aria-invalid` указывает ошибочный ввод. `aria-describedby` дополняется идентификатором ошибки, существующая подсказка сохраняется. Фокус ставим только после ответа с ошибками, а не при каждом символе.

Проверка

Пустая регистрация: фокус в логине, текст ошибки рядом с полем, счётчик 0 из 6. Исправьте ввод и отправьте снова.

Регистрация с ошибками

Конференции.РФ

[Помещения](#) [Войти](#) [Регистрация](#)

Создайте учётную запись

Все поля обязательны.

Заполнение формы
Заполнено 0 из 6 обязательных полей. Проверка выполняется при отправке.

Логин *

Латиница и цифры: 6-30 символов
Латиница и цифры: 6-30 символов.

ФИО *

Кириллица и пробелы
Укажите ФИО кириллицей.

Телефон *

8(XXX)XXX-XX-XX
Формат: 8(XXX)XXX-XX-XX.

Электронная почта *

Укажите адрес почты.

Пароль *

От 8 до 128 символов
Пароль: от 8 до 128 символов.

Повторите пароль *

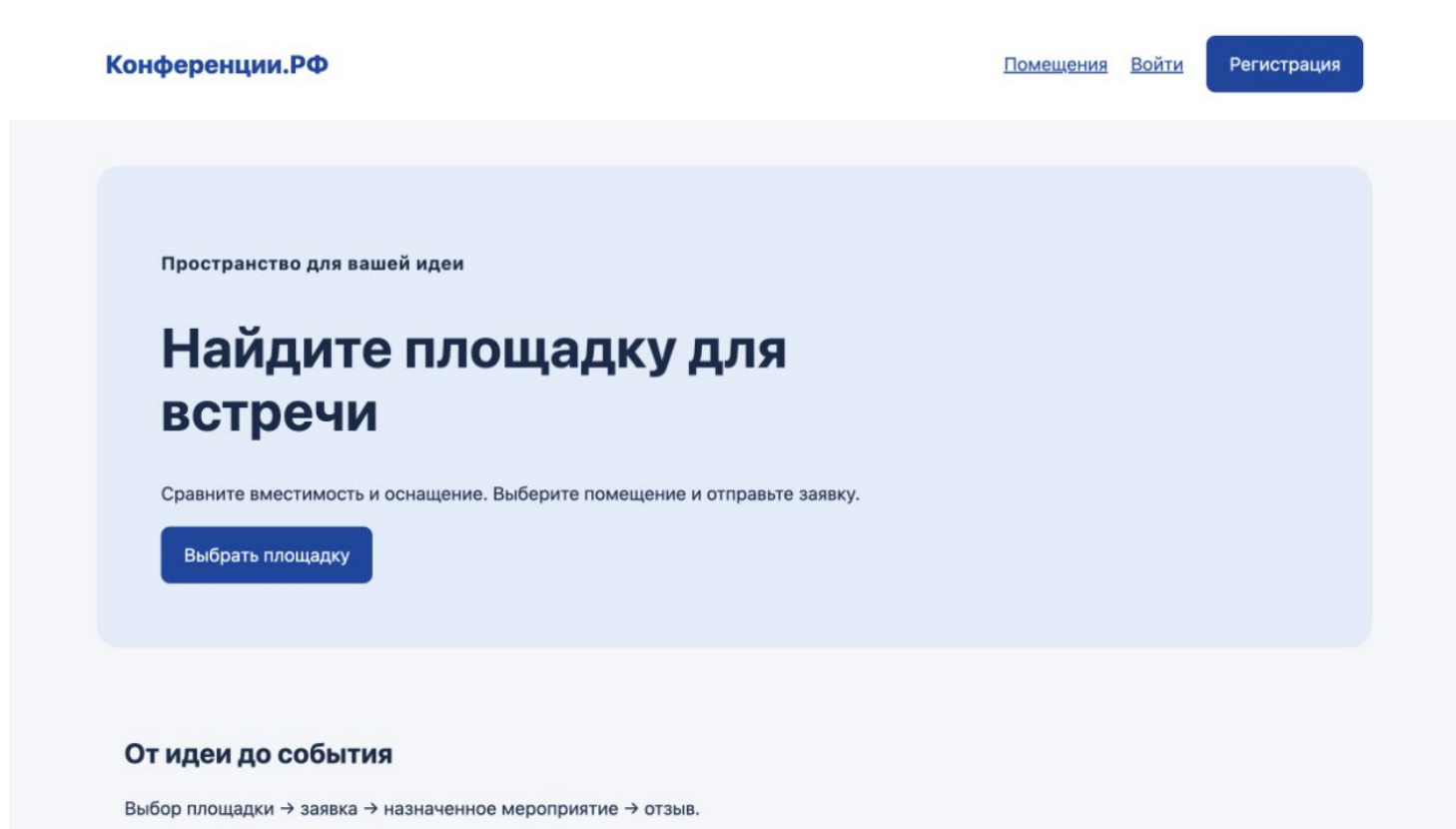
[Создать пользователя](#)

Конференции.РФ · Учебный пример ПМ.08

Снимок созданного по фрагментам примера: сервер объясняет ошибку, интерфейс помогает её исправить.

Сопоставить готовый экран с проектным решением

Главная страница после оформления



Одно главное действие ведёт к каталогу; текст объясняет, что произойдёт далее.

Обосновать результат

Сравните экран с исходным бумажным каркасом. Покажите, где пользователь понимает назначение, находит главное действие и видит следующий шаг. Проверьте контраст и читаемость, а не только декоративное сходство.

Проверить пустой фильтр и мобильный экран

Пустой результат фильтра

[Конференции.РФ](#)[Помещения](#)[Войти](#)[Регистрация](#)

Помещения для вашей конференции

Тип

Город

Не менее мест

Все типы

1000

Показать

[Сбросить](#)

Найдено: 0

Подходящих помещений не нашлось

Измените условия или [сбросьте фильтры](#).

Конференции.РФ · Учебный пример ПМ.08

При вместимости 1000 есть объяснение и сброс условий.

Регистрация 390×844

Конференции.РФ

[Помещения](#) [Войти](#) [Регистрация](#)

Создайте учётную запись

Все поля обязательны.

Заполнение формы
Заполнено 0 из 6 обязательных полей. Проверка выполняется при отправке.

Логин *

Латиница и цифры: 6–30 символов

ФИО *

Кириллица и пробелы

Телефон *

8(XXX)XXX-XX-XX

Мобильная форма созданного примера сохраняет подписи, подсказку и доступность полей.

Проверка

Просмотрите все страницы при 390×844. Пройдите Tab и Shift+Tab, исправьте ошибку, отправьте заявку. Поля не выходят за экран; длинный текст переносится.

Создать решение своей предметной области

Данные, а не готовое приложение

В разделе предметных областей доступен только JSON с учебными данными. Возьмите структуру и приёмы, которые вы самостоятельно реализовали по методичке. Создайте новую пустую БД с другим именем; дайте учебному пользователю права на неё, измените MYSQL_DATABASE и один раз выполните init-db и import-rooms для JSON своей области. Так данные примера не смешаются с вариантом.

Самостоятельный дизайн

Измените название сервиса, типы ресурсов в TYPES, тексты и пользовательские действия по смыслу области. Разработайте свою композицию, палитру и типографику. Для изображений разместите выбранные вами файлы в static/img и исправьте пути image. Если фотографии пока нет, укажите пустую строку, чтобы работала заглушка.

Дополнительные задания

Выполните оба условия своей области. Для каждого опишите проблему пользователя, место изменения, внесённый код и фактическую проверку. Например, оснащение помогает оценить пригодность ресурса; перенос длинного названия позволяет прочитать карточку на мобильном экране.

Проверить работу и объяснить решения

Функциональный путь

Регистрация с ошибкой и успехом → вход → фильтр и пустой результат → заявка → свои записи → вход администратора → оба статуса → отзыв. Отдельно проверьте 403 для обычного пользователя на /admin, чужой отзыв, повторный отзыв и POST без csrf. Ошибочное действие не должно менять данные.

Типичные причины ошибки

TemplateNotFound: проверьте имя созданного шаблона и папку templates. BuildError: сопоставьте имя функции маршрута и url_for. Соединение MySQL: проверьте переменные и запущенный сервер. CSS 404: файл должен быть static/design.css. Не повторяйте импорт в заполненную базу ради обновления внешнего вида.

Отчёт и Git

В общем шаблоне отчёта покажите собственные решения и фактические проверки. Снимки должны отражать вашу предметную область. При сдаче проекта приложите ссылку на его репозиторий. Не включайте секреты, виртуальное окружение, сессии и реальные пользовательские данные.